GTSFM Studio Dask execution profiles verification

TECHNICAL VERIFICATION REPORT | 24 OCTOBER 2026

Source and evidence commit: 8dafa2ded1a43344f0203e867e640b1292f51516

Saved, validated Dask profiles now persist across Studio restarts and expose requested versus observed worker settings. On the bounded RTX 3080 experiment, two manual runs and two persisted-profile runs exported byte-identical cameras, sparse points, observations and retrieval pairs: four registered cameras and 1,509 tracks in every run. This verifies the configuration capability and output preservation on the tested case, without establishing a speedup.

Hypothesis and claim boundary

A named, host-local Studio execution profile should survive process restart, validate resource settings before launch, resolve to the same runner arguments as the established manual controls, and report what Dask actually launched. This is an engineering repeatability claim. It does not predict an algorithmic speedup, enforce a hard GPU-VRAM quota, or establish heterogeneous or multi-GPU scaling.

The implementation uses schema version 1 and stores named profiles at `~/.gtsfm/studio-execution-profiles.json` by default. `GET /api/execution-profiles` lists profiles; `PUT /api/execution-profiles/{name}` creates or updates one; and `DELETE /api/execution-profiles/{name}` removes one. Profile selection on `POST /api/jobs` is authoritative. Invalid integer values (including booleans and fractions), unsupported modes or GPU IDs, relative scratch paths, malformed stores, and conflicting manual controls fail before launch. Each run records the resolved settings in `execution_profile.json`. Dask snapshots reuse `gtsfm/utils/dask_stats.py` and include aggregate counts plus worker PID, thread count, host-memory limit, scratch path, CUDA visibility, and Torch device identity.

Configuration and browser evidence

The independent configuration report configuration run 9 (raw/independent-verifier/config-run-9-current/configuration.json) exercised four fresh HTTP server processes and an injected temporary store. It reports `VERIFIED` for persistence after restart, invalid-setting rejection, corrupt-store fail-closed behavior, local-only remote boundaries, conflicting run controls, cross-process persistence, and exact manual/profile argv and environment equivalence. The concurrent profile writes and fresh-process reads are retained in that report. The telemetry contract preserves `CUDA_VISIBLE_DEVICES` unset (`null`) versus explicitly empty (`""`); an environment string alone is not treated as proof of GPU assignment.

The real browser workflow artifacts fullstack-ui-result.json (raw/fullstack-ui-result.json) and fullstack-restart-result.json (raw/fullstack-restart-result.json) show profile creation and update through FastAPI, dirty-draft protection, validation errors, reload, and persistence in a fresh process. A user workflow is: open GTSFM Studio, choose **Compute this machine**, open **Machine settings**, save or select a named profile, then start a run and inspect the run details for requested versus observed worker, thread, host-memory, scratch, and GPU values. `npm run check` and `npm run build` passed for the frontend. The existing manual and Modal paths remain available.

The RTX 3080 preflight reports 10,240 MiB total VRAM and a successful CUDA tensor operation in rtx3080_nvidia_smi.txt (raw/rtx3080_nvidia_smi.txt) and rtx3080_torch_probe.txt (raw/rtx3080_torch_probe.txt). The guarded importable-file Dask probe rtx3080_dask_probe.txt (raw/rtx3080_dask_probe.txt) observed one process worker, one thread, a 1,000,000,000-byte host-memory limit, an actual worker scratch directory, and Torch CUDA visibility of an NVIDIA GeForce RTX 3080. This early probe records accessibility but has no independent post-close PID

receipt; the later measured trials provide that cleanup evidence. A retained stdin-based probe failed because multiprocessing spawn cannot import <stdin>; the guarded-file harness resolved that harness limitation.

Matched RTX 3080 experiment

The experiment protocol was frozen in series-declaration.json (raw/gpu-experiment/series-declaration.json) before the measured outputs. The apparatus used the first four real Lund Door JPEGs (DSC_0001.JPG through DSC_0004.JPG) from the pinned 12-image source, MegaLoc retrieval, SIFT/TwoWayMatcher, vanilla VGGT, and GTSAM refinement. Each trial used one Dask process worker, two threads, an 18 GiB worker host-memory limit, explicit CUDA_VISIBLE_DEVICES=0, resolution 518, crop mode, isolated scratch and empty per-trial source/cache directories. Startup random, NumPy, Torch, and geometry seeds were 42; Dask's worker pre-spawn hash seed was 6640 and is recorded by role. The source, weights, DINOv2, environment, argv, resolved Hydra configuration, output hashes, process observations, cgroup limits, telemetry, and cleanup checks are retained under gpu-experiment (raw/gpu-experiment/README.md).

The two-arm design was manual baseline repeats A1/A2 versus the saved-profile path through the production JobManager.start library API in B1/B2. These GPU trials exercised the same profile-resolved launch path without driving the launch through browser HTTP; the separate browser and HTTP persistence workflow is covered above. The four completed runs are:

Trial	Launch path	Harness wall (s)	Logged pipeline (s)	Peak GPU used (MiB)	Result
A1	Manual runner	47.725	31.3	6,708	exit 0
B1	Saved profile to JobManager to runner	44.915	28.9	5,586	exit 0
A2	Manual runner	43.332	28.8	5,596	exit 0
B2	Saved profile to JobManager to runner	43.653	25.1	6,796	exit 0

The measured values are descriptive: there are two repeats per arm, harness wall time includes launch, polling, artifact copy, and a fixed seven-second passive settling interval (removed from the reported wall value), and pipeline time comes from unmodified logs. The spread does not support a speedup claim. The full machine-readable table is measured-series-summary.json (raw/gpu-experiment/measured-series-summary.json).

Reconstruction preservation and independent comparison

The predeclared acceptance protocol (dask_profiles_experiment.md) requires baseline A1/A2 repeatability and exact image, pair, camera-metadata, track-observation and color identities. After one orientation-preserving Sim(3) fit from at least three noncollinear camera centers, every camera center must differ by at most 1e-5 * D, every orientation by 0.001 degree, every sparse point by 1e-4 * D, and every exported track reprojection error by 0.011 pixel; intrinsics use atol=rtol=1e-6. Here D is the baseline camera-center bounding-box diagonal. Nonfinite values fail; thresholds cannot be widened after observing outputs.

All four runs produced four registered cameras and 1,509 final sparse tracks. The retrieved pair identities and the three exported COLMAP files (cameras.txt, images.txt, and points3D.txt) are byte-identical in every arm; hashes are listed in gpu-export-hashes.json (raw/independent-verifier/gpu-export-hashes.json). The independent comparison gpu-four-trial-comparison.json (raw/independent-verifier/gpu-four-trial-comparison.json) is VERIFIED for its predeclared exported-sparse-reconstruction equivalence claim. It checks registered-image identities, track observations and colors,

camera metadata and intrinsics, finite/nondegenerate geometry, Sim(3)-aligned camera centers and points, and exported reprojection error. The largest reported aligned errors are 2.54e-17 camera-center-over-diameter, 2.42e-6 degrees rotation, 3.57e-15 point-over-diameter, and 0 px exported reprojection error. Exact byte equality is the decisive criterion here, so it preserves point IDs and observation multiplicity as well as visible geometry.

The verifier also retained a deliberate failed identity attempt in gpu-canonical-ambiguity.json (raw/independent-verifier/gpu-canonical-ambiguity.json): observation-only canonicalization found 77 duplicate rounded observation groups (1,431 distinct groups versus 1,509 tracks). That result remains `INCONCLUSIVE` for nonidentical ambiguous exports; it was not reclassified as a pass. The stronger full-file and pair-byte comparison resolves this ambiguity for the four identical outputs without loosening a tolerance. This distinction matters if a future run produces different but geometrically similar files.

The frozen four-trial series used the accepted feature source. The independent controls audit gpu-controls-audit.json (raw/independent-verifier/gpu-controls-audit.json) confirms matching argv and resolved Hydra configuration apart from per-run output roots, fixed inputs/model/source/cache/worker controls, and cleanup observations. Separate final-source follow-ups F1 and F2 reran the profile path after the final telemetry fallback; F2 uses telemetry source hash `524f5100c19c5c26f0b389481179f8fbdd1ec1cd4ccb7a6ce00e245c3d3d8046` with profile source hash `b7623a40098a3fdf60d9b0991bc31428a1018e985c6cd7ba42dc80e7c3b8520a`. Their source declarations, patches, summaries and matching export hashes are under final-source (raw/gpu-experiment/final-source/). These are final-source smoke confirmations, not extra matched repeats and not a change to the frozen A1/B1/A2/B2 statistics.

The committed telemetry file has SHA256 `4b0fa22c14ba3835871f415e414a8a1cefff17c252623c977adca7972f2f11c4` after a required blank-line formatting fix. Its parsed Python AST is exactly equal to the F2-tested `524f5100...` file. Coordinator validation (raw/coordinator-validation.json) records this check, a separate rerun of the four-trial comparison, 64 passing focused backend/runtime tests, the frontend type check and production build, applicable CI lint, and the corrected test-environment setup failure. No full-repository test-suite pass is claimed.

Cancellation and cleanup

The persisted-profile cancellation receipt under initial/cancel1 (raw/gpu-experiment/initial/cancel1/) records a real `JobManager.cancel` during active Dask reconstruction. The runner returned `-15`; all 37 observed runner, worker, compiler and resource-tracker PIDs were absent afterward, and the GPU compute-process query was empty. A resource-tracker warning about six semaphore objects was retained in the raw logs. The evidence therefore supports process and GPU cleanup after cancellation; it does not support a warning-free or complete IPC-cleanup claim. The final drift report also confirms no trial PID or experiment unit remained and that the pre-existing MIRA service was left alive.

Verdict and limitations

The profile engineering claim is **VERIFIED** for the tested macOS and Linux/WSL local filesystem paths using POSIX advisory locking (`fcntl`) plus in-process locking, API/UI round trip, fresh-process persistence, strict validation, manual/profile argument equivalence, observed one-worker RTX 3080 launch, exact four-trial export preservation, and cancellation process/GPU cleanup boundaries. The latest independent configuration run and CPU null-versus-empty checks are retained under independent-verifier (raw/independent-verifier/). Native Windows and filesystems without the supported POSIX locking behavior remain outside this evidence. The reconstruction result supports equivalence for this bounded four-image dataset and this pinned source/model/configuration. It does not establish full 12-image scalability, algorithmic performance improvement, hard VRAM enforcement, multi-GPU or heterogeneous scaling, warning-free IPC cleanup, or behavior for untested execution modes. The 12-image preflight reached near physical VRAM capacity and is retained as a capacity observation, not as a measured series or scalability claim.

Reproduction and evidence

To reproduce the measured package, use the exact guarded harness and service envelope in gpu-experiment/README.md (raw/gpu-experiment/README.md), with a fresh isolated base and unique trial names. Do not run the harness from Python stdin because multiprocessing spawn requires an importable file. The source copy on the RTX 3080 host has no repository `.git`; the retained 852-file manifest, source diff/state, pinned VGGT revision and weight digests are the provenance boundary. No global environment, public service, network grant, or pre-existing MIRA service was modified.

The pinned official VGGT source revision is `44b3afbd1869d8bde4894dd8ea1e293112dd5eba`. Its 5,026,874,952-byte weight file has SHA256 `d15bf50a8615c8225ed48b51ea5cac673d82442ec0309036df555a053253afe0`, downloaded from `facebook/VGGT-1B` revision `860abec7937da0a4c03c41d3c269c366e82abdf9`. All four input JPEG hashes and the separate MegaLoc/DINOv2 provenance are in the declaration and environment manifest. Model and input files are referenced by immutable hashes rather than bundled as gigabyte copies.

On the retained Linux compute host, the following reruns the frozen manual arm with a new trial ID; use `R2` and `--arm profile` for the persisted-profile arm. Reusing an existing trial ID is refused. This assumes the retained source, dataset and model assets described in the raw README; it does not provision a fresh host.

```
systemd-run --user --unit=gtsfm-profile-R1 --wait --collect \
  -p MemoryMax=24G -p MemorySwapMax=0 \
  -p CPUQuota=400% -p RuntimeMaxSec=960 \
  --setenv=PATH=/home/jasdeep/gtsfm_eval/.venv/bin:/usr/local/bin:/usr/bin:/bin:/usr/lib/wsl/lib \
  /home/jasdeep/gtsfm_eval/.venv/bin/python \
  /home/jasdeep/gtsfm-profile-gpu-20261024/experiment_dask_profiles_gpu.py \
  --base /home/jasdeep/gtsfm-profile-gpu-20261024 \
  --trial R1 --arm manual \
  --dataset /home/jasdeep/gtsfm-profile-gpu-20261024/dataset-first4 \
  --wall-seconds 900
```

The local comparison tool can verify the retained outputs without a GPU. Run `python scripts/verify_dask_profiles.py compare --help`; supply the four `merged_retriangulated` directories and named-pair files in **A1, A2, B1, B2** order (comparison argument order differs from execution order), and an output JSON path. The canonical machine-readable verdicts are claims.json (raw/independent-verifier/claims.json); the raw experiment has a 562-artifact checksum manifest and the independent audit has a separate 101-file index.

This work used AI-assisted engineering under human-directed scope. The implementation worker integrated the backend and runner changes; separate UI, execution, and verification workers supplied disjoint artifacts; the coordinator reviewed the evidence, ran independent comparisons and tests, and owns final publication. Automated and peer-agent checks are reported as evidence with their raw inputs; they are not a substitute for a human review of deployment suitability.